

INTEGRATED TECHNICAL RESUME

지원자 직무경험 기술서

Systems & AI Full-Stack Engineer

컴퓨터/소프트웨어공학 전공

2026.07 졸업 예정

OS 커널, 컴파일러, Spring Boot 백엔드, Vue/React Native 클라이언트, FastAPI AI 추론 서버, Docker/AWS 배포까지 직접 구현하며 낮은 레벨의 시스템 이해와 제품 전체 흐름을 함께 다루는 엔지니어로 성장해 왔습니다. 문제를 증상 수준에서 봉합하기보다 실행 순서, 상태 전이, API 규약, 데이터 흐름, 배포 제약까지 내려가 원인을 좁힙니다. 어떤 조직에서도 기초가 탄탄하고, 끝까지 디버깅하며, 문서로 협업하는 개발자로 기여할 수 있습니다.

핵심 지표

92.52% ECG 진단 모델 Macro AUC

TPS 66 AWS 환경 주요 API 처리량

0건 API 응답 규약 파싱 오류 수렴

4개 완수한 핵심 기술 프로젝트

기술 스택

SYSTEMS

C / C++ · MIPS Assembly · OS Kernel · Compiler · Virtual Memory · IPC · File System

BACKEND / DATA

Java · Spring Boot 3 · PostgreSQL · Elasticsearch · Redis · MinIO · REST API

FRONTEND / MOBILE

Vue 3 · TypeScript · React Native · API Integration · UI State

AI / INFRA / COLLABORATION

Python · FastAPI · PyTorch · Docker · AWS EC2 · GitHub Actions · RFC Docs

학력 · 어학

학력 소프트웨어공학 학사
2020.09 – 2026.07 졸업 예정

중국어 TSC Level 6 · 기술적 회화 가능

영어 OPIc IH · 기술 문서 독해 및 협업 가능

한국어

지원 적합성

- 낮은 레벨부터 제품 레벨까지 연결: 커널 IPC, 컴파일러 파서, 백엔드 API, AI 추론 서버, 모바일 앱까지 직접 다루며 추상화 아래에서 문제가 생기는 방식을 경험했습니다.
- 범용성이 높은 기술 기반: C/C++ 시스템, Java 백엔드, 데이터베이스, 프론트엔드, AI 서버, 클라우드 배포 경험을 갖춰 다양한 엔지니어링 포지션에 대응할 수 있습니다.
- 데이터와 문서 기반 협업: 기술 선택과 팀 내 이견을 감정이 아닌 벤치마크, RFC, 실행 문서로 정리해 합의 속도와 재현성을 높인 경험이 있습니다.

핵심 역량

구조적으로 문제를 줄이는 디버깅

IPC 레이스 컨디션, 파서 에러 복구, API 응답 불일치, EC2 OOM처럼 반복 가능한 문제를 실행 순서, 상태, 공통 계층, 배포 전략 관점으로 해결했습니다.

제품 전체 흐름을 보는 구현 경험

Spring Boot API, Vue/React Native 클라이언트, FastAPI AI 서버, Docker 배포를 연결해 사용자 입력부터 서버 응답, 데이터 저장, 배포 제약까지 함께 고려합니다.

명확한 문서화와 다국어 협업

문제가 생기면 재현 조건과 로그를 먼저 고정하고, 해결 과정을 RFC와 README로 남깁니다. 한·중·영 환경에서 기술 문서를 읽고 팀 맥락을 맞출 수 있습니다.

업무 방식

- 문제를 받으면 먼저 입력, 상태, 출력, 경계 조건으로 나누고 가장 작은 재현 케이스를 만듭니다.
- 기술 선택이 갈릴 때는 벤치마크와 사용자 영향도를 함께 제시해 제품 판단으로 연결합니다.
- 구현 후에는 예외 케이스, 성능 수치, API 규약, 배포 가능성을 함께 확인해 실제 제품에 들어갈 수 있는 형태로 마무리합니다.

직무경험 기술서

Projects 01 / 02 · CS Fundamentals & Systems

01 MIPS 마이크로커널 운영체제

2024.03 – 2024.07 · 1인 개발

OS Kernel 전체 구현 · 메모리/프로세스/IPC/파일 시스템 · Lock-free IPC race condition 해결

MIPS 아키텍처 위에서 부팅, 가상 메모리, 프로세스, 시스템콜, IPC, 파일 시스템까지 OS 핵심 구성요소를 직접 구현하며 시스템 소프트웨어와 백엔드 장애 분석에 필요한 내부 동작 감각을 쌓았습니다.

- 구현**
- 2단계 페이지 테이블과 TLB Fast Refill 핸들러를 어셈블리로 최적화하고, Copy-on-Write 기반 `fork()` 를 구현해 프로세스 생성 비용을 낮췄습니다.
 - 사용자 공간 FS 서버와 공유 페이지 기반 Pipe IPC를 구현해 커널과 파일 시스템을 분리하고, 시스템콜 흐름을 직접 설계했습니다.
- 문제**
- Pipe IPC 구현 중 스케줄링 타이밍에 따라 열린 파이프를 닫힌 것으로 오인하는 동시성 문제가 발생했습니다. 단순히 인터럽트를 막는 방식보다 근본 원인인 **매핑 순서 의존성**을 분석했습니다.
- 해결**
- `close()` 와 `dup()` 호출 시 페이지 map/unmap 순서를 재배치해 `pageref(fd) ≤ pageref(pipe)` 불변식이 항상 유지되도록 설계했습니다. 락을 추가하지 않고 실행 순서 재배치로 안정성을 확보했습니다.
- 연결점**
- 자료구조, 메모리 관리, 프로세스, 파일 I/O, 동시성 제어를 한 시스템 안에서 구현했습니다. 라이브 코딩과 CS 심화 인터뷰에서 깊게 설명할 수 있는 대표 프로젝트입니다.

C · MIPS Assembly · GxEmul Simulator · Microkernel FS · Pipe IPC · Virtual Memory · File System

02 SysY 언어 컴파일러 프론트엔드 & VM

2024.10 – 2025.01 · 1인 개발

C++17/CMake · Lexer/Parser/Symbol Table/P-Code VM · Fault-Tolerant 파서

SysY 언어의 Lexer, 재귀 하향 Parser, Symbol Table, P-Code VM 전 과정을 C++17과 CMake로 구현하고, 구문 오류가 발생해도 파싱을 중단하지 않는 결함 허용 구조를 설계했습니다.

- 구현**
- 모호성을 해결하는 재귀 하향 파서를 설계하고, 스코프 체인을 지원하는 심볼 테이블로 함수 시그니처 불일치와 중복 선언을 컴파일 타임에 진단했습니다.
 - 단일 파일 구조를 Lexer, Parser, SymbolTable, ErrorHandler, Interpreter 5개 모듈로 분리하고 CMake 기반 빌드 흐름으로 정리했습니다.
- 문제**
- 세미콜론 누락 에러 복구 시 토큰 포인터가 과도하게 이동해 정상 구문까지 연쇄적으로 오류로 판정하는 문제가 발생했습니다.
- 해결**
- 에러를 파싱 중단 사유가 아닌 **문맥 복귀 신호**로 재정의하고, 직전 유효 토큰 위치인 `mLastNonT` 를 기록해 파서를 중단하지 않고 정상 흐름으로 복구하도록 개선했습니다.
- 연결점**
- 알고리즘과 자료구조를 실제 컴파일러 파이프라인에 녹였고, 비정상 입력에서도 시스템 상태를 잃지 않는 구현 원칙을 익혔습니다. 빠른 디버깅과 라이브 코딩 역량을 보여 주기 좋습니다.

C++17 · CMake · Lexer / Parser · P-Code VM · Symbol Table · Error Recovery

직무경험 기술서

Projects 03 / 04 · Product Engineering & AI

03 학술 성과 공유 플랫폼 — 백엔드/프론트 통합

2025.03 – 2025.08 · 백엔드 전담

5인 팀 프로젝트 · Spring Boot/Vue 연동 · ES/Redis/MinIO 통합 · API 오류 0건 수렴

Vue 3 프론트와 Spring Boot 백엔드, PostgreSQL, Elasticsearch, Redis, MinIO를 연결해 검색·상세 조회·분석·외부 API 연동이 가능한 학술 성과 플랫폼을 구축했습니다.

- 구현**
- Spring Boot 3(Java 21) 환경에서 PostgreSQL, Elasticsearch, Redis, MinIO를 연동하고 Vue 3 + TypeScript 프론트와 API 통신 구조를 맞췄습니다.
 - 검색 성능 요구에 맞춰 RDBMS와 Elasticsearch를 함께 운용하고, Redis 캐시와 MinIO 파일 저장 흐름을 분리해 데이터 경계를 명확히 했습니다.
- 문제**
- 팀원마다 API 응답 구조가 달라 프론트엔드 파싱 오류가 반복됐고, 로컬 환경 설정 차이로 인프라 기동 실패가 발생했습니다. 단순 문서 합의만으로는 재발을 막기 어려웠습니다.
- 해결**
- Spring의 `ResponseBodyAdvice` 로 컨트롤러 반환값을 항상 `{code, message, data}` 구조로 래핑하고, Docker Compose와 README/RFC로 실행 조건을 표준화했습니다.
- 연결점**
- 여러 데이터 소스와 프론트엔드 UI가 맞물리는 제품에서 중요한 API 계약, 상태 일관성, 개발환경 표준화 경험을 보여 줍니다.

Java · Spring Boot 3 · Vue 3 · TypeScript · PostgreSQL · Elasticsearch · Redis · MinIO · Docker Compose

04 CareLink — 모바일·백엔드·AI 추론 플랫폼

2025.10 – 2026.03 · 1인 개발 / 졸업논문

React Native · Spring Boot · FastAPI · AWS EC2/Docker · ECG AUC 92.52% · TPS 66

모바일 앱, Spring Boot 백엔드, FastAPI AI 추론 서버를 분리한 3-Tier 구조를 설계하고 AWS EC2와 Docker 기반으로 배포해 실제 사용 흐름과 운영 제약을 함께 검증했습니다.

- 구현**
- React Native 앱에서 Spring Boot API와 FastAPI 추론 서버를 연결하고, JWT 인증·자동 로그아웃·Cache-First 조회 전략으로 사용자 흐름을 개선했습니다.
 - PTB-XL 기반 ECG 진단 모델을 FastAPI 추론 API로 제공하고, 20명 동시 접속 기준 주요 API TPS 56.23~66.41을 측정했습니다.
- 문제**
- AI 컨테이너와 백엔드를 동시에 구동하면 1GB RAM EC2에서 OOM이 반복됐습니다. 서버에서 직접 이미지를 빌드하는 방식은 배포 중 서비스 안정성을 떨어뜨렸습니다.
- 해결**
- 로컬과 CI에서 이미지를 빌드해 레지스트리에 push하고, 서버는 Pull-only로 운영하도록 전환했습니다. Docker `mem_limit` 과 `profile` 로 컨테이너별 자원을 격리했습니다.
- 연결점**
- 클라이언트·백엔드·AI 서버·배포 제약이 함께 맞물리는 제품 문제를 끝까지 연결해 본 경험입니다. AI 기능이 포함된 서비스, 모바일 앱, 백엔드 플랫폼 포지션에 모두 활용할 수 있습니다.

React Native · Spring Boot 3 · FastAPI · Python · PyTorch · PostgreSQL · AWS EC2 · Docker · GitHub Actions